

PRD - Symulator pracy silnika czterosuwowego w OpenGL

1. Nazwa projektu

Interaktywny symulator pracy silnika czterosuwowego R4

Pełny temat:

Symulator pracy silnika czterosuwowego. Widoczne ruchome tłoki, zawory i wał korbowy. Projekt zawiera animację, interakcję użytkownika, teksturowanie, oświetlenie oraz hierarchiczny model 3D silnika.

2. Cel projektu

Celem projektu jest stworzenie interaktywnej sceny 3D w OpenGL przedstawiającej uproszczony, ale czytelny model silnika czterosuwowego. Scena ma pokazywać mechanikę pracy silnika: ruch tłoków, obrót wału korbowego, pracę korbowodów oraz otwieranie i zamykanie zaworów ssących i wydechowych.

Projekt ma spełniać wymagania laboratoriów z grafiki komputerowej: użycie OpenGL i GLM, animacja/interakcja, kilka poruszających się obiektów 3D, tekstury, minimum dwa źródła światła, cieniowanie oraz nietrywialne modele 3D. Projekt nie powinien korzystać ze starego stylu OpenGL typu glBegin, glVertex, glRotate, glTranslate, glPushMatrix; zamiast tego należy używać m.in. glDrawArrays/glDrawElements, glVertexAttribPointer oraz transformacji z GLM.

3. Zakres projektu

3.1. Zakres podstawowy

Projekt musi zawierać:

1. Scenę 3D z modelem silnika R4, czyli czterocylindrowego silnika rzędowego.

2. Widoczne elementy:

- blok silnika w formie przekroju,
- 4 cylindry,
- 4 tłoki,
- 4 korbowody,
- wał korbowy,
- zawory ssące i wydechowe,
- głowicę silnika,
- prosty układ dolotowy i wydechowy,
- koło zamachowe albo koło pasowe.

3. Animację:

- obracający się wał korbowy,
- tłoki poruszające się góra/dół,
- korbowody pracujące razem z tłokami,
- zawory otwierające się zgodnie z fazą pracy cylindra.

4. Interakcję:

- zwiększanie/zmniejszanie obrotów silnika,
- pauza/wznowienie animacji,
- zmiana widoku kamery,
- obracanie/oddalanie kamery.

5. Cieniowanie:

- minimum dwa źródła światła,
- widoczne różnice jasności na obiektach.

6. Teksturowanie:

- minimum dwie różne tekstury,
- np. metal, ciemny metal, guma, szkło/panel zegarów.

7. Panel informacyjny/HUD:

- aktualne RPM,
- poziom gazu,
- opis aktualnego suwu dla każdego cylindra.

4. Założenia technologiczne

Projekt powinien być wykonany w C++ z użyciem:

OpenGL
GLFW
GLEW albo GLAD - zależnie od szkieletu projektu
GLM
stb_image albo LodePNG do tekstur
Assimp opcjonalnie do modeli OBJ

Jeżeli projekt bazuje na szkielecie z laboratoriów, należy zachować styl pracy z macierzami P, V, M, ponieważ prezentacja laboratoryjna pokazuje taki model potoku graficznego: macierz modelu, widoku i rzutowania, czyli:

$$a' = P * V * M * a$$

Transformacje obiektów mają być realizowane przez GLM:

```
glm::translate(...)
glm::rotate(...)
glm::scale(...)
glm::lookAt(...)
glm::perspective(...)
```

Rysowanie geometrii powinno używać:

```
glDrawArrays(...)
glDrawElements(...)
glVertexAttribPointer(...)
glEnableVertexAttribArray(...)
```

5. Wymagania zgodności z zasadami projektu

Wymaganie	Sposób spełnienia
OpenGL + GLM	Cała scena oparta na OpenGL, transformacje w GLM
Kilka obiektów 3D	Silnik składa się z wielu części: tłoki, zawory, wał, blok, korbwody

Obiekty poruszające się niezależnie	Tłoki, zawory, wał i koło zamachowe mają własne animacje
Interakcja lub animacja	Gaz, kamera, pauza, reset
Tekstury	Metal, guma, panel zegarów, blok silnika
Minimum 2 światła	Światło kierunkowe + punktowe
Nietrywialne modele	Własne meshe albo importowane OBJ
Brak starego OpenGL	Zakaz glBegin, glEnd, glVertex, glRotate, glTranslate, glPushMatrix
Brak różowego	Nie używać koloru różowego

Zasady oceniania premiuja przede wszystkim: nietrywialne modele, poprawne zarządzanie obiektami na scenie, animację, kamerę, teksturowanie i cieniowanie. Ten projekt jest pod to dobrze ustawiony, bo mechanika silnika naturalnie wymusza ruch wielu części i hierarchiczne transformacje.

6. Model funkcjonalny silnika

6.1. Typ silnika

Projekt przedstawia uproszczony silnik:

R4 - cztery cylindry w rzędzie
 Silnik czterosuwowy
 Cykl pracy: 720 stopni obrotu wału

6.2. Suwy silnika

Każdy cylinder przechodzi przez 4 suwy:

Zakres kąta	Suw	Tłok	Zawór ssący	Zawór wydechowy
0-180 stopni	ssanie	w dół	otwarty	zamknięty
180-360 stopni	sprężanie	w górę	zamknięty	zamknięty
360-540 stopni	praca	w dół	zamknięty	zamknięty
540-720 stopni	wydech	w górę	zamknięty	otwarty

6.3. Przesunięcie faz cylindrów

Każdy cylinder ma własne przesunięcie fazy:

```
phaseOffset[0] = 0.0f;
phaseOffset[1] = 180.0f;
phaseOffset[2] = 360.0f;
phaseOffset[3] = 540.0f;
```

Alternatywnie można później zastosować kolejność zapłonu 1-3-4-2, ale na potrzeby projektu najważniejsza jest czytelność animacji.

7. Wymagania funkcjonalne

FR-01 - Scena 3D

Program powinien tworzyć okno OpenGL i wyświetlać scenę 3D z silnikiem w centrum.

FR-02 - Kamera

Użytkownik może zmieniać widok kamery.

Minimalne widoki:

- 1 - widok izometryczny
- 2 - widok z boku
- 3 - widok z góry
- 4 - widok z przodu

Dodatkowo:

- A/D - obrót kamery wokół silnika
- Q/E - zoom

FR-03 - Animacja wału korbowego

Wał korbowy obraca się stale wokół własnej osi. Prędkość obrotu zależy od aktualnej wartości RPM.

FR-04 - Animacja tłoków

Każdy tłok porusza się pionowo zgodnie z kątem wału i przesunięciem fazowym cylindra.

Przykładowy wzór:

```
float pistonY = baseY + stroke * 0.5f * (1.0f + cos(angle));
```

FR-05 - Animacja korbowodów

Korbowod powinien wizualnie łączyć punkt na wale korbowym z tłokiem. Może być uproszczony jako cylinder lub wydłużony prostokąt ustawiony między dwoma punktami.

FR-06 - Animacja zaworów

Każdy cylinder ma zawór ssący i wydechowy.

Zawory przesuwają się w dół/górę zależnie od aktualnego suwu:

```
ssanie    -> zawór ssący otwarty
wydech    -> zawór wydechowy otwarty
pozostałe -> oba zamknięte
```

FR-07 - Gaz / throttle

Użytkownik może zwiększać lub zmniejszać gaz.

Sterowanie:

```
W - zwiększ gaz
S - zmniejsz gaz
```

Gaz wpływa na RPM:

```
targetRpm = minRpm + throttle * (maxRpm - minRpm);
```

RPM powinno dochodzić do wartości docelowej płynnie, a nie skokowo.

FR-08 - Pauza

```
SPACE - pauza/wznowienie animacji
R      - reset animacji
```

FR-09 - HUD

Na ekranie powinien być widoczny panel informacyjny:

```
RPM: 2500
Throttle: 64%
Cylinder 1: Intake
Cylinder 2: Compression
Cylinder 3: Power
Cylinder 4: Exhaust
```

HUD może być wykonany jako prosty overlay tekstowy albo jako panel 3D obok silnika, np. z zegarami.

FR-10 - Tryb edukacyjny

Opcjonalnie użytkownik może włączyć opisy faz pracy cylindrów:

- H - pokaż/ukryj HUD
- F - pokaż/ukryj kolory faz pracy

Kolory faz:

- ssanie - niebieski
- sprężanie - szary
- praca - pomarańczowy
- wydech - czerwony

Nie używać różowego.

8. Wymagania нефункционалне

NFR-01 - Czytelność kodu

Kod powinien być podzielony na moduły. W wariantcie niezależnym można użyć takiej struktury:

```
main.cpp
Engine.h / Engine.cpp
Cylinder.h / Cylinder.cpp
Mesh.h / Mesh.cpp
Shader.h / Shader.cpp
Texture.h / Texture.cpp
Camera.h / Camera.cpp
```

Jeżeli bazujemy na paczce laboratoryjnej, ta struktura może zostać dopasowana do istniejących plików. Szczegóły są w sekcji 26.

NFR-02 - Brak starego OpenGL

Nie wolno używać:

```
glBegin
glEnd
glVertex
glNormal
glTexCoord
glRotate
glTranslate
glScale
gluLookAt
gluPerspective
glPushMatrix
glPopMatrix
```

NFR-03 - Wydajność

Projekt powinien działać płynnie na typowym komputerze laboratoryjnym.

Cel:

```
minimum 30 FPS
preferowane 60 FPS
```

NFR-04 - Możliwość demonstracji

Projekt musi być łatwy do pokazania prowadzącemu. Po uruchomieniu od razu powinno być widać działający silnik.

NFR-05 - Znajomość kodu

Każdy ważny fragment kodu powinien być zrozumiały i możliwy do wyjaśnienia podczas oddawania projektu. Szczególnie: animacja, macierze, shadery, tekstury, cieniowanie i sposób rysowania meshy.

9. Architektura programu

9.1. Główne klasy

Application

Odpowiada za:

- inicjalizację OpenGL
- pętlę programu
- obsługę wejścia
- wywołanie update i render

EngineSimulator

Odpowiada za logikę całego silnika:

```
class EngineSimulator {
public:
    void update(float deltaTime);
    void draw(const glm::mat4& view, const glm::mat4& projection);

    void increaseThrottle();
    void decreaseThrottle();
    void togglePause();
    void reset();

private:
    float crankAngle;
    float rpm;
    float targetRpm;
    float throttle;
    bool paused;

    std::vector<Cylinder> cylinders;
};
```

Cylinder

Odpowiada za jeden cylinder:

```
class Cylinder {
public:
    Cylinder(int index, float phaseOffset, glm::vec3 position);

    void update(float crankAngle);
    void draw(const glm::mat4& parentModel);

    float getPistonY() const;
```

```
StrokeType getStrokeType() const;
```

```
private:
    int index;
    float phaseOffset;
    glm::vec3 position;

    float pistonY;
    float intakeValveOffset;
    float exhaustValveOffset;
    StrokeType stroke;
};
```

Mesh

Odpowiada za dane geometrii:

```
class Mesh {
public:
    void uploadToGPU();
    void draw();

private:
    std::vector<Vertex> vertices;
    std::vector<unsigned int> indices;

    GLuint vao;
    GLuint vbo;
    GLuint ebo;
};
```

Shader

Odpowiada za shadery:

```
class Shader {
public:
    void use();
    void setMat4(const std::string& name, const glm::mat4& value);
    void setVec3(const std::string& name, const glm::vec3& value);
    void setFloat(const std::string& name, float value);
    void setInt(const std::string& name, int value);
};
```

Camera

Odpowiada za widok:

```
class Camera {
public:
    glm::mat4 getViewMatrix() const;

    void rotate(float deltaAngle);
    void zoom(float delta);
    void setPreset(CameraPreset preset);
};
```

10. Dane i stany

10.1. Enum suwu

```
enum class StrokeType {
    Intake,
    Compression,
    Power,
    Exhaust
};
```

10.2. Stan silnika

```
struct EngineState {
    float crankAngleDegrees;
    float rpm;
    float targetRpm;
    float throttle;
    bool paused;
};
```

10.3. Stan cylindra

```
struct CylinderState {
    float phaseAngleDegrees;
    float pistonY;
    float intakeValveOffset;
    float exhaustValveOffset;
    StrokeType stroke;
};
```

11. Algorytm aktualizacji animacji

11.1. Aktualizacja czasu

W każdej klatce:

```
float currentTime = glfwGetTime();
float deltaTime = currentTime - previousTime;
previousTime = currentTime;
```

11.2. Aktualizacja RPM

```
targetRpm = minRpm + throttle * (maxRpm - minRpm);
rpm += (targetRpm - rpm) * smoothing * deltaTime;
```

Przykładowe wartości:

```
minRpm = 700.0f;
maxRpm = 6000.0f;
smoothing = 3.0f;
```

11.3. Aktualizacja kąta wału

RPM oznacza obroty na minutę.

```
float revolutionsPerSecond = rpm / 60.0f;
float degreesPerSecond = revolutionsPerSecond * 360.0f;
```

```
crankAngleDegrees += degreesPerSecond * deltaTime;
crankAngleDegrees = fmod(crankAngleDegrees, 720.0f);
```

Używamy zakresu 0-720, bo cykl czterosuwowy trwa dwa obroty wału.

11.4. Aktualizacja cylindra

Dla każdego cylindra:

```
float phase = fmod(crankAngleDegrees + phaseOffset, 720.0f);
```

Wyznaczanie suwu:

```
if (phase >= 0 && phase < 180) stroke = Intake;
else if (phase >= 180 && phase < 360) stroke = Compression;
else if (phase >= 360 && phase < 540) stroke = Power;
else stroke = Exhaust;
```

Pozycja tłoka:

```
float angleRad = glm::radians(phase);
pistonY = baseY + strokeLength * 0.5f * (1.0f + cos(angleRad));
```

Zawory:

```
intakeValveOffset = (stroke == StrokeType::Intake) ? openOffset : 0.0f;
exhaustValveOffset = (stroke == StrokeType::Exhaust) ? openOffset : 0.0f;
```

12. Model sceny

12.1. Lista obiektów

Obiekt	Liczba	Animowany	Teksturowany	Uwagi
Blok silnika	1	nie	tak	przekrój
Cylinder	4	nie	tak	tuleje cylindrów
Tłok	4	tak	tak	ruch góra/dół
Korbowód	4	tak	tak	łączy tłok z wałem
Wał korbowy	1	tak	tak	obrót
Zawór ssący	4	tak	tak	otwieranie na ssaniu
Zawór wydechowy	4	tak	tak	otwieranie na wydechu
Koło zamachowe	1	tak	tak	obraca się z wałem
Układ dolotowy	1	nie	tak	prosty kolektor
Układ wydechowy	1	nie	tak	prosty kolektor
HUD / zegary	1	tak	opcjonalnie	pokazuje RPM

13. Hierarchia transformacji

Projekt powinien korzystać z hierarchicznych macierzy modelu. To jest mocny punkt projektu, bo silnik naturalnie składa się z elementów zależnych od siebie.

Przykład struktury:

```
M_engine
├─ M_block
├─ M_crankshaft
│   └─ M_flywheel
├─ M_cylinder_1
│   └─ M_piston_1
│   └─ M_connecting_rod_1
```

```

|   ├── M_intake_valve_1
|   └── M_exhaust_valve_1
└── M_cylinder_2
    ├── M_cylinder_3
    └── M_cylinder_4

```

Dzięki temu można łatwo obracać cały silnik, a części wewnętrzne zachowują własny ruch lokalny.

14. Geometria i modele

14.1. Minimalne podejście

Można stworzyć własne proste meshe:

```

cube/prostopadłościan
cylinder
sphere/okrągły tłok
rod/cylinder między punktami
torus/koło zamachowe

```

14.2. Lepsze podejście

Użyć modeli OBJ:

```

piston.obj
valve.obj
crankshaft.obj
engine_block.obj
connecting_rod.obj

```

Kod ładowania i rysowania modelu musi być zrozumiały i obsługiwany przez nas. Można użyć biblioteki do wczytania modelu, ale kod rysujący powinien być własny i oparty o OpenGL.

15. Tekstury

15.1. Wymagane tekstury

Minimum:

```

metal.png
dark_metal.png
rubber.png
gauge.png

```

15.2. Mapowanie tekstur

Każdy główny obiekt powinien mieć współrzędne teksturowania.

Dla obiektów proceduralnych:

```

struct Vertex {
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 texCoord;
};

```

15.3. Przypisanie tekstur

Obiekt	Tekstura
łłoki	metal.png
wał	dark_metal.png
korbowody	metal.png
blok silnika	dark_metal.png
zawory	metal.png
pasek/element gumowy	rubber.png
zegary	gauge.png

16. Oświetlenie

16.1. Źródła światła

Minimum dwa światła:

- Directional Light - światło główne z góry/przodu
- Point Light - światło punktowe nad silnikiem

16.2. Model cieniowania

Preferowany model:

Phong albo Blinn-Phong

Minimalnie:

Lambert + ambient + specular

Shader powinien obsługiwać:

```
uniform vec3 lightDir;
uniform vec3 pointLightPos;
uniform vec3 viewPos;

uniform vec3 ambientColor;
uniform vec3 diffuseColor;
uniform vec3 specularColor;
```

17. Shadery

17.1. Vertex shader

Wejścia:

```
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aNormal;
layout(location = 2) in vec2 aTexCoord;
```

Uniformy:

```
uniform mat4 P;
uniform mat4 V;
uniform mat4 M;
```

Wyjścia:

```
out vec3 FragPos;
```

```
out vec3 Normal;
out vec2 TexCoord;
```

17.2. Fragment shader

Wejścia:

```
in vec3 FragPos;
in vec3 Normal;
in vec2 TexCoord;
```

Uniformy:

```
uniform sampler2D texture0;
uniform vec3 viewPos;
uniform vec3 dirLightDirection;
uniform vec3 pointLightPosition;
```

Wyjście:

```
out vec4 FragColor;
```

18. Sterowanie

Klawisz	Akcja
W	zwiększ gaz
S	zmniejsz gaz
SPACE	pauza/wznowienie
R	reset animacji
A	obrót kamery w lewo
D	obrót kamery w prawo
Q	oddal kamerę
E	przybliż kamerę
1	widok izometryczny
2	widok z boku
3	widok z góry
4	widok z przodu
H	pokaż/ukryj HUD
F	pokaż/ukryj kolory faz cylindrów
ESC	wyjście

19. Etapy realizacji

Etap 1 - Szkielet projektu

Cel:

Uruchomienie okna OpenGL, kamery, shaderów, Z-bufora i podstawowej pętli renderującej.

Zakres:

```
main.cpp / main_file.cpp
Shader / ShaderProgram
Camera
basic mesh drawing
perspective projection
depth test
```

Kryterium ukończenia:

Na ekranie widać prosty obiekt 3D z kamerą i światłem.

Etap 2 - System meshy

Cel:

Przygotowanie własnej klasy Mesh z VAO/VBO/EB0 albo wykorzystanie istniejącej klasy Model jako bazy.

Zakres:

```
Vertex structure
Mesh::uploadToGPU()
Mesh::draw()
createCube()
createCylinder()
createRod()
```

Kryterium ukończenia:

Można narysować kilka różnych obiektów przez glDrawElements albo glDrawArrays.

Etap 3 - Model jednego cylindra

Cel:

Zbudowanie jednego działającego cylindra z tłokiem, korbowodem, zaworami i fragmentem wału.

Zakres:

```
Cylinder class
piston movement
valve movement
crank rotation
connecting rod transform
```

Kryterium ukończenia:

Jeden cylinder pracuje w cyklu czterosuwowym.

Etap 4 - Silnik R4

Cel:

Rozbudowa jednego cylindra do czterech cylindrów.

Zakres:

```
4 cylinder instances
phase offsets
shared crankshaft
flywheel
engine block
```

Kryterium ukończenia:

Cztery tłoki pracują z różnymi fazami, a wał obraca się wspólnie.

Etap 5 - Interakcja

Cel:

Dodanie sterowania gazem, pauzą, resetem i kamerą.

Zakres:

```
keyboard callback
throttle
rpm smoothing
camera presets
```

pause/reset

Kryterium ukończenia:

Użytkownik może wpływać na pracę silnika i widok.

Etap 6 - Tekstury

Cel:

Dodanie tekstur na główne obiekty.

Zakres:

Texture class albo readTexture z LodePNG
texture loading
UV coordinates
minimum 2 textures

Kryterium ukończenia:

Obiekty są otekstutowane, a scena nie jest jednokolorowa.

Etap 7 - Cieniowanie

Cel:

Dodanie widocznego oświetlenia z minimum dwóch źródeł.

Zakres:

Phong/Blinn-Phong shader
directional light
point light
normal vectors
specular highlights

Kryterium ukończenia:

Cieniowanie jest widoczne na metalu, wale, tłokach i bloku.

Etap 8 - HUD i efekty edukacyjne

Cel:

Dodanie informacji o stanie silnika.

Zakres:

RPM display
throttle display
stroke names
phase colors
optional combustion flash
optional exhaust particles

Kryterium ukończenia:

Widać, w jakiej fazie znajduje się każdy cylinder.

Etap 9 - Finalizacja

Cel:

Przygotowanie projektu do oddania.

Zakres:

README
opis sterowania

opis architektury
 czyszczenie kodu
 usunięcie martwych plików
 test build

Kryterium ukończenia:

Projekt da się szybko uruchomić i łatwo pokazać prowadzącemu.

20. Definition of Done

Projekt uznajemy za gotowy, jeżeli:

- ☐ Uruchamia się bez błędów
- ☐ Scena zawiera silnik R4
- ☐ Wał korbowy się obraca
- ☐ Tłoki poruszają się góra/dół
- ☐ Korbowody łączą wał z tłokami
- ☐ Zawory otwierają się zgodnie z fazą pracy
- ☐ Jest gaz i RPM
- ☐ Jest pauza i reset
- ☐ Kamera jest sterowalna
- ☐ Są minimum 2 tekstury
- ☐ Są minimum 2 źródła światła
- ☐ Cieniowanie jest widoczne
- ☐ Kod nie używa zakazanych funkcji starego OpenGL
- ☐ Projekt nie używa koloru różowego
- ☐ Jest README z instrukcją uruchomienia i sterowania

21. Ryzyka

Ryzyko	Skutek	Obejście
Za trudny model wału korbowego	Opóźnienie projektu	Użyć uproszczonego wału z cylindrów i przesuniętych czopów
Trudne ustawienie korbowodu między punktami	Brzydka animacja	Na start użyć prostopadłościanu obracanego między dwoma punktami
Problemy z OBJ	Brak modeli	Zacząć od proceduralnych brył
HUD tekstowy trudny w OpenGL	Strata czasu	Zrobić HUD jako obiekty 3D obok silnika
Cieniowanie źle wygląda	Mniej punktów	Użyć prostego Phong'a i poprawnych normalnych
Za duża liczba szczegółów	Niedokończenie	Najpierw MVP: wał + 4 tłoki + zawory

22. MVP projektu

Minimalna wersja do pokazania:

Silnik R4 w przekroju
 4 tłoki
 4 korbowody
 obracający się wał
 zawory ssące i wydechowe
 sterowanie RPM
 kamera

2 tekstury
2 światła
cieniowanie

Bez MVP nie dodawać bajerów.

23. Rozszerzenia po MVP

Po działającym MVP można dodać:

iskra/spalanie w cylindrze
dym z wydechu
kolorowe fazy pracy
panel zegarów 3D
tryb slow motion
przezroczysty blok silnika
animowany pasek/koło pasowe
lepsze modele OBJ
normal mapy/specular mapy

24. Prompt dla Codexa

Ten fragment można wkleić bezpośrednio do Codexa:

Pracujemy nad projektem C++ OpenGL/GLM: interaktywny symulator pracy silnika czterosuwowego R4. Projekt musi używać nowoczesnego OpenGL: VAO/VBO/EB0, glDrawArrays albo glDrawElements, glVertexAttribPointer, shaderów GLSL oraz transformacji przez GLM. Nie wolno używać glBegin/glEnd/glVertex/glRotate/glTranslate/glScale/glPushMatrix/glPopMatrix/gluLookAt/gluPerspective.

Celem jest scena 3D przedstawiająca przekrojowy silnik R4. Widoczne elementy: blok silnika, 4 cylindry, 4 tłoki, 4 korbowody, wał korbowy, zawory ssące i wydechowe, koło zamachowe oraz prosty HUD. Silnik ma być animowany: wał się obraca, tłoki chodzą góra/dół, korbowody łączą tłoki z wałem, zawory otwierają się zależnie od fazy czterosuwu. Cykl trwa 720 stopni. Fazy: 0-180 ssanie, 180-360 sprężanie, 360-540 praca, 540-720 wydech. Każdy cylinder ma przesunięcie fazy.

Dodaj interakcję: W/S zmienia gaz i RPM, SPACE pauzuje, R resetuje, A/D obraca kamerę, Q/E zoom, 1-4 zmieniają preset kamery, H ukrywa/pokazuje HUD. Scena ma mieć minimum dwie tekstury i minimum dwa źródła światła. Cieniowanie powinno być widoczne, najlepiej Phong albo Blinn-Phong.

Proszę generować projekt etapami:

1. szkielet OpenGL z oknem, kamerą, shaderem i Z-buforem,
2. klasy Shader, Camera, Mesh, Texture,
3. proceduralne meshe: cube, cylinder, simple rod,
4. klasa EngineSimulator i Cylinder,
5. animacja jednego cylindra,
6. rozbudowa do 4 cylindrów,
7. tekstury,
8. światła i cieniowanie,
9. HUD i finalizacja.

Kod ma być czytelny, modularny i łatwy do wyjaśnienia prowadzącemu.

25. Krótki opis projektu do README

Symulator pracy silnika czterosuwowego R4

Projekt z grafiki komputerowej przedstawiający interaktywny symulator pracy czterosuwowego silnika rzędowego R4. Scena 3D pokazuje przekrój silnika z ruchomymi tłokami, korbowodami, zaworami oraz obracającym się wałem korbowym.

Funkcje

- animowany wał korbowy,
- ruchome tłoki,
- ruchome korbowody,
- zawory ssące i wydechowe zależne od fazy pracy,
- cykl czterosuwowy 720 stopni,
- regulacja gazu i RPM,
- kamera sterowana klawiaturą,
- tekstury materiałów,
- oświetlenie z minimum dwóch źródeł,
- cieniowanie Phong/Blinn-Phong,
- panel informacyjny z RPM i fazami cylindrów.

Sterowanie

W/S - gaz większy/mniejszy
 SPACE - pauza
 R - reset
 A/D - obrót kamery
 Q/E - zoom
 1/2/3/4 - presety kamery
 H - HUD
 ESC - wyjście

26. Dopasowanie do paczki laboratoryjnej bez zmiany koncepcji PRD

Ta sekcja nie zmienia projektu. Ona tylko mówi, jak realizować powyższe PRD w strukturze paczki z laboratoriów.

26.1. Co już mamy w paczce laboratoryjnej

W paczce startowej znajdują się m.in.:

```
main_file.cpp
shaderprogram.cpp / shaderprogram.h
model.cpp / model.h
cube.cpp / cube.h
sphere.cpp / sphere.h
torus.cpp / torus.h
teapot.cpp / teapot.h
allmodels.h
lodepng.cpp / lodepng.h
v_*.glsl oraz f_*.glsl
bricks_diffuse.png
tester.png
Visual Studio solution/project
GLEW, GLFW, GLM
```

Wniosek: nie trzeba pisać wszystkiego od zera. Można potraktować paczkę jako bazę startową i stopniowo dodawać własne pliki projektu.

26.2. Proponowana struktura plików w ramach paczki

Nie nadpisujemy sensu PRD. Tylko dopasowujemy nazwy do startera:

gkiw_st_02_win/	
main_file.cpp	- wejście programu, pętla, callbacki, wywołanie silnika
shaderprogram.*	- zostaje jako bazowa obsługa shaderów
model.*	- może zostać jako baza/wspólny model
lodepng.*	- ładowanie tekstur
EngineSimulator.h/.cpp	- logika całego silnika
Cylinder.h/.cpp	- logika pojedynczego cylindra
EngineMeshes.h/.cpp	- proceduralne meshe: cylinder, tłok, korbówód, wał
EngineRenderer.h/.cpp	- rysowanie silnika, wysyłanie macierzy i tekstur
CameraController.h/.cpp	- kamera orbitująca i presety
TextureManager.h/.cpp	- uchwytury tekstur, jeżeli nie wystarczy readTexture
v_engine.glsl	- docelowy vertex shader
f_engine.glsl	- docelowy fragment shader
textures/	
metal.png	
dark_metal.png	
rubber.png	
gauge.png	

26.3. Jak mapować stare PRD na paczkę

Element z PRD	Realizacja w paczce lab
Application	funkcje w main_file.cpp
Shader	istniejący ShaderProgram
Texture	lodepng + funkcja readTexture
Mesh	istniejący Model albo nowa klasa/struktura na VAO/VBO/EBO
Camera	nowy CameraController albo zmienne globalne kamery w main_file.cpp na MVP
EngineSimulator	nowy plik EngineSimulator.h/.cpp
Cylinder	nowy plik Cylinder.h/.cpp
drawEngine()	nowy renderer albo funkcje pomocnicze w main_file.cpp na start

26.4. Czego nie robić przy pracy z paczką

Nie przebudowywać od razu całej paczki na wielką architekturę. Najbezpieczniej:

1. Najpierw uruchomić starter bez zmian.
2. Dodać jeden plik EngineSimulator i wywołać go z drawScene.
3. Dopiero potem wydzielać Cylinder, CameraController, EngineMeshes.
4. Nie usuwać istniejących shaderów, tylko dodać nowe v_engine.glsl i f_engine.glsl.
5. Nie opierać finalnej sceny wyłącznie na Models::cube, Models::sphere, Models::torus. Mogą pomóc na MVP, ale finalnie główne elementy silnika powinny być własnymi meshami albo importowanymi modelami.

26.5. Prompt dla Codexa z uwzględnieniem paczki lab

Bazujemy na paczce laboratoryjnej gkiw_st_02_win. Nie zmieniamy koncepcji projektu z PRD: nadal robimy interaktywny symulator pracy silnika czterosuwowego R4. Pamiętaj, że istnieją już pliki main_file.cpp, shaderprogram.*, model.*, cube/sphere/torus/teapot, lodepng.*, shadery v_*.glsl/f_*.glsl oraz biblioteki GLEW/GLFW/GLM.

Nie przepisuj projektu od zera. Najpierw przeanalizuj `main_file.cpp` i sposób rysowania z użyciem `ShaderProgram`, macierzy `P/V/M`, `GLM` oraz `glDrawArrays/glDrawElements`. Następnie dodawaj projekt etapami.

Docelowo dodaj nowe moduły:

- `EngineSimulator.h/.cpp`
- `Cylinder.h/.cpp`
- `EngineMeshes.h/.cpp`
- `EngineRenderer.h/.cpp`, jeśli będzie potrzebny
- `CameraController.h/.cpp`, jeśli będzie potrzebny
- `v_engine.glsl`
- `f_engine.glsl`

Na MVP możesz korzystać z istniejących prostych modeli jako placeholderów, ale finalnie główne elementy silnika powinny być rysowane jako własne `meshe` albo importowane modele. Nie używaj `glBegin/glEnd/glVertex/glRotate/glTranslate/glScale/glPushMatrix/glPopMatrix`. Transformacje tylko przez `GLM`. Rysowanie przez `glDrawArrays/glDrawElements`.

Zacznij od etapu 1: uruchom istniejący starter i dodaj minimalny `EngineSimulator`, który rysuje jeden cylinder z animowanym tłokiem i wałem.